fastml: Guarded Resampling Workflows for Safer Automated Machine Learning in R

by Selcuk Korkmaz, Dincer Goksuluk, and Eda Karaismailoglu

Abstract Preprocessing leakage arises when scaling, imputation, or other data-dependent transformations are estimated before resampling, inflating apparent performance while remaining hard to detect. We present fastml, an R package that emphasizes leakage-aware execution through guarded resampling, where preprocessing is re-estimated inside each resample and then applied to the corresponding assessment data. The package supports grouped and time-ordered resampling, blocks high-risk configurations such as full-analysis splits, and audits recipes for dependencies on external data or environments. We evaluate fastml with a Monte Carlo simulation contrasting global and fold-local normalization, a usability comparison with tidymodels under matched specifications, and survival benchmarks across datasets of different sizes. Across 100 simulation runs, global preprocessing yielded a mean ROC AUC of 0.809 (SD 0.018) versus 0.651 (SD 0.046) under guarded execution, an average inflation of 0.158 (95% CI 0.149 to 0.167). fastml matched held-out performance obtained with tidymodels while reducing workflow orchestration, and it supported consistent benchmarking of multiple survival model classes through a unified interface. fastml is an R package whose execution model is designed to make leakage-aware evaluation easier to implement correctly in R.

1 Introduction

Over the past decade, machine learning workflows in R have evolved from monolithic interfaces, such as **caret** (Kuhn, 2008), toward modular ecosystems including **tidymodels** (Kuhn and Wickham, 2020) and **mlr3** (Lang et al., 2019). These frameworks decompose modeling into explicit steps for resampling, preprocessing, model specification, and evaluation, providing flexibility and methodological transparency. However, this modularity also shifts responsibility to the user to correctly assemble and coordinate multiple components into a valid pipeline. This shift increases the risk of methodological error by expanding the number of decisions that must be coordinated correctly. Users must ensure that preprocessing, resampling, and model fitting are applied in the proper order and with strict separation between training and evaluation data. In complex workflows, particularly those involving grouped or nested resampling, even experienced practitioners may inadvertently introduce errors that are not detected by standard software checks.

Among these errors, data leakage remains one of the most consequential. In particular, preprocessing leakage occurs when transformations such as scaling, imputation, or feature construction are estimated using information that should be unavailable at evaluation time, for example by learning transformations on the full dataset prior to resampling. This violates training-test isolation and can substantially inflate performance estimates while remaining difficult to detect, because models may still appear well-validated under standard cross-validation (Hastie et al., 2009). Recent studies report that leakage and related evaluation flaws are widespread in applied machine learning and contribute to irreproducible and overly optimistic results across multiple scientific domains (Kapoor and Narayanan, 2023; Rosenblatt et al., 2024; Tampu et al., 2022).

Modern R frameworks provide tools that can prevent preprocessing leakage, most notably by coupling transformations to resampling-aware workflows. However, these safeguards are typically optional rather than enforced. Correct usage requires users to explicitly define recipes or workflows, ensure that preprocessing is trained within each resampling split, and avoid global data-dependent transformations prior to model evaluation. When these conditions are not met, incorrect pipelines can run without warnings while yielding

biased estimates. As a result, methodological validity often depends more on user discipline than on guarantees provided by the software. For example, tidymodels' `fit_resamples()` and `tune_grid()` correctly re-estimate preprocessing within each fold when a recipe is embedded in a workflow; however, users must assemble this workflow correctly, and nothing prevents them from applying global preprocessing before invoking these functions. fastml addresses this by making fold-local preprocessing the default execution path and by adding structural checks — such as full-analysis split detection and recipe environment scanning — that are not present in general-purpose frameworks.

The **fastml** package (Korkmaz et al., 2025) addresses this gap through a safety-by-design interface that emphasizes leakage-aware execution patterns while retaining access to established modeling engines. A central mechanism is its guarded resampling path, in which preprocessing is re-estimated within each resample on the analysis split and the fold-trained transformations are then applied to the corresponding assessment split. **fastml** also includes checks intended to surface common configuration errors, including aborting a resampling run when a full-analysis (no-holdout) split is detected and rejecting user recipes that appear to depend on external data sources. When hyperparameter tuning is requested under this guarded path, tuning is performed inside the same resampling loop.

More broadly, **fastml** provides resampling options and diagnostics that target additional evaluation pitfalls. Grouped resampling can keep related observations together when repeated measures or clustered records exist. For time-ordered settings, blocked or rolling resampling relies on an explicit ordering variable and **fastml** warns when the ordering required by the chosen design is not provided. An optional audit mode can flag risky patterns in custom preprocessing, such as references to the global environment or code that reads data from files or writes outputs to files, which can create hidden dependencies and undermine reproducibility.

These mechanisms are best understood as leakage-risk mitigation under supported configurations rather than universal guarantees; users can still introduce leakage upstream, for example by supplying preprocessed inputs or externally constructed resamples. The contribution of **fastml** is therefore not a new learning algorithm, but an execution model that makes leakage-aware evaluation easier to carry out correctly in practice by packaging common patterns, defaults, and checks into a single interface. This paper describes the design and implementation of **fastml**, details how guarded resampling and related checks are realized when those execution paths are used, and demonstrates via simulation how preprocessing leakage can inflate performance estimates and how fold-local workflow fitting mitigates this failure mode under the evaluated configurations.

The remainder of this paper focuses on the leakage-mitigation architecture and its empirical evaluation; additional utilities — parallel execution, bootstrap confidence intervals, adaptive classification thresholds, tuning complexity presets, and post hoc model explanation via `fastexplain()` — are documented in the package.

2 Materials and methods

2.1 Guarded resampling

Preprocessing leakage arises when data-dependent transformation parameters are estimated using information that should be unavailable at evaluation time (Kapoor and Narayanan, 2023; Kaufman et al., 2012). This most commonly occurs when steps such as scaling, imputation, or dimensionality reduction are learned on the full dataset prior to resampling (Vabalas et al., 2019). Because transformation parameters are estimated using all rows before the data are split into folds, observations that later belong to assessment folds contribute to those parameters, which are then applied within each training fold, breaking training-test separation and tending to yield optimistic performance estimates (Hastie et al., 2009). This failure mode does not require access to outcome labels: even unsupervised transformations can leak information when their parameters are estimated globally (Kaufman et al., 2012).

The general problem of optimistic evaluation due to leakage from preprocessing and evaluation design has been widely discussed in the applied machine learning literature (Hastie et al., 2009; Kaufman et al., 2012).

Modern R frameworks provide the capability to avoid preprocessing leakage by embedding transformations within resampling-aware workflows. However, this isolation is typically optional rather than enforced, and leaky pipelines can run without warnings while yielding biased estimates if preprocessing is applied globally before resampling. Figure 1 illustrates the distinction. In a leaky workflow, transformation parameters are estimated on the full dataset before resampling, so each fold's assessment data influences the transformation applied to that fold's training data. In a properly specified workflow, transformation parameters are estimated using only the training portion of each fold and then applied to that fold's assessment portion.

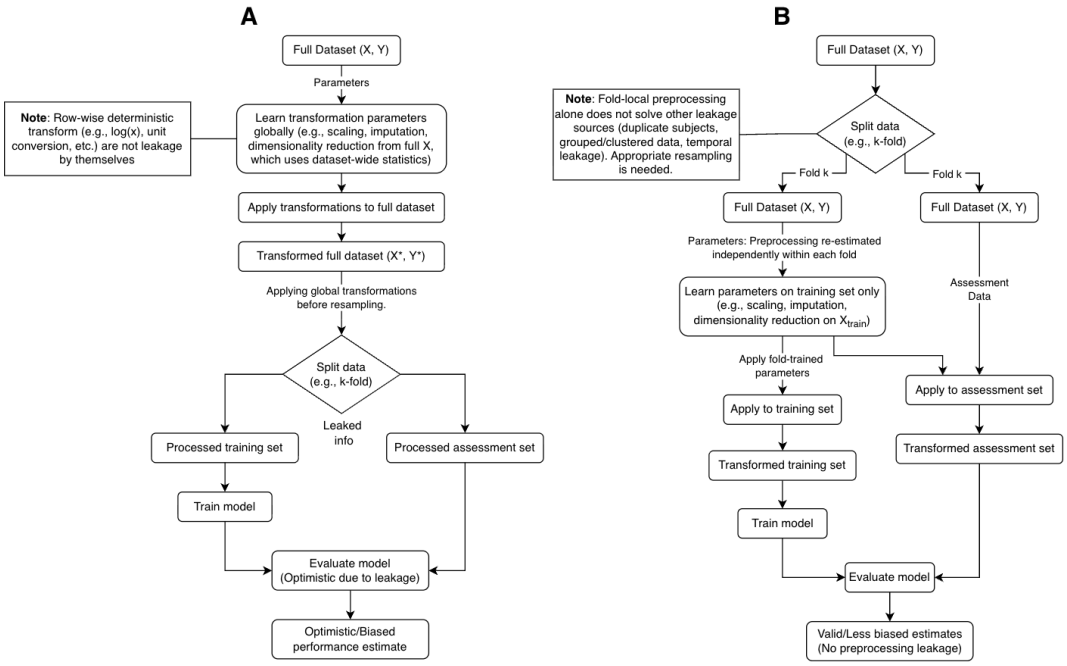


Figure 1: Illustration of a leaky cross-validation workflow caused by global preprocessing.

When **fastml** executes its guarded resampling path (i.e., when workflows are fitted through `fastml_guarded_resample_fit`), it organizes preprocessing and model fitting fold-by-fold to reduce preprocessing-leakage risk (Section 2.1).

The guarded architecture

Under guarded resampling, any data-dependent preprocessing is re-estimated within each resample. For each fold k, the procedure is:

- **Generate split** k: Construct a resampling split and obtain an analysis (training) set $D^k_{analysis}$ and an assessment (hold-out) set D^k_{assess}.
- **Fit fold-specific preprocessing**: Fit a fresh preprocessing specification $R^{(k)}$ using only $D^{(k)}_{analysis}$. This includes estimating any data-dependent transformation parameters (e.g., imputation values, scaling parameters, feature construction rules) from the fold's analysis data.
- **Apply fold-trained preprocessing**: Apply $R^{(k)}$ to $D^{(k)}_{analysis}$ to produce the preprocessed training features, and apply the same fold-trained preprocessing $R^{(k)}$ to $D^{(k)}_{assess}$ to

produce the corresponding preprocessed assessment features. The assessment set is not used when fitting $R^{(k)}$; it is only transformed after $R^{(k)}$ has been learned from the analysis data.

- **Fit and evaluate within the fold**: Train the model on the preprocessed $D^{(k)}_{analysis}$ and evaluate it on the transformed $D^{(k)}_{assess}$, producing performance for fold k.

Repeating these steps across folds ensures that, within the resampling process, data-dependent preprocessing parameters are estimated without access to the corresponding assessment data.

Implementation in fastml

In **fastml**, the guarded resampling behavior is implemented in the resampling engine used when the package executes its guarded resampling path. The core implementation is `fastml_guarded_resample_fit`, which iterates directly over **rsample** (Frick et al., 2025) split objects. For each split k, the guarded resampling engine: - Extracts the analysis (training) and assessment (hold-out) portions of the raw data for that split. - Fits the user-defined workflow, including preprocessing via **recipes** (Kuhn et al., 2025), on the analysis portion of the split. - Applies the fitted preprocessing to the assessment portion and evaluates the trained model on that fold's assessment data.

This structure ensures that, within the guarded resampling loop, recipes are trained on the analysis portion of each split and then applied to the corresponding assessment portion for evaluation. The guard also performs a structural check to detect pathological resampling definitions. In particular, it detects "full-analysis" splits, cases where the analysis indices cover the full dataset and therefore no holdout remains, and stops with an error when such a configuration is encountered. This check is implemented by inspecting resample indices (the `in_id` index sets), rather than by comparing full data objects directly. As noted in Section 1, upstream preprocessing or externally constructed resamples remain outside the scope of these guards.

In addition to the resampling structure, **fastml** includes a recipe-scanning guard that is not a full static analyzer and does not attempt to detect every potentially unsafe operation (e.g., arbitrary function calls, or code that reads data from files or writes outputs to files). Instead, it targets a narrower class of high-impact problems: it scans recipe steps for references to external environments or global objects (e.g., `.GlobalEnv`, parent frames, or environment inheritance patterns) and flags steps that appear to embed data objects (for example, steps that inherit from `data.frame`). Recipes flagged as depending on external environments or embedded data objects are rejected before model training begins, with an informative message. This reduces the risk of recipes implicitly depending on global state or carrying fold-external data inside step definitions, but it does not guarantee detection of all possible leakage or reproducibility hazards.

Taken together, **fastml**'s guarded resampling functionality reduces leakage risk by (i) fitting workflows within each resample when executed through `fastml_guarded_resample_fit`, (ii) detecting full-analysis resampling failures by checking `in_id` indices, and (iii) blocking recipe steps that appear to reference external environments or embed external data objects (Section 2.1).

Beyond preprocessing leakage, **fastml** also supports resampling designs that reduce common evaluation artifacts. Grouped cross-validation keeps related rows together to reduce group-based leakage (Kuhn and Johnson, 2013), and blocked/rolling resampling modes surface warnings that correct ordering is required to avoid training on future information (Hyndman and Athanasopoulos, 2018).

Resampling design for grouped and time-ordered data

Fold-local preprocessing addresses a specific leakage mechanism: estimating data-dependent transformation parameters using information that should be unavailable at evaluation time. However, valid evaluation can still fail when the resampling design does not match the dependence structure of the data. Two common cases are grouped (clustered) data and time-ordered data.

In grouped or clustered settings, optimistic evaluation can occur even when preprocessing is strictly fold-local. If correlated records from the same entity (e.g., repeated measurements from a patient, multiple samples from the same subject, or clustered observations from the same unit) are split across folds, models can exploit within-entity similarity and appear to generalize better than they truly do (Roberts et al., 2017). To reduce this risk, **fastml** supports grouped resampling designs in which all rows sharing a grouping identifier are assigned to the same split, using `group_cols` under the corresponding grouped resampling method. This can mitigate group-based leakage when the grouping variable captures the relevant dependence structure (Section 2.1).

In time-ordered settings, leakage can arise when the resampling scheme allows training data to include observations occurring after the assessment period, producing "future-to-past" contamination. This can occur without any explicit preprocessing mistake, purely because the split violates the temporal direction implied by the prediction task (Bergmeir and Benítez, 2012). **fastml** supports blocked and rolling resampling modes intended for such settings, which rely on an explicit ordering variable (e.g., `block_col`) and associated block/window parameters. Because correct ordering is a prerequisite for valid time-aware evaluation, **fastml** warns when the ordering required by the chosen resampling design is not provided. As with grouped designs, these options mitigate a common leakage route under appropriate configuration (Section 2.1) (Hyndman and Athanasopoulos, 2018).

These resampling designs therefore complement fold-local preprocessing: guarded resampling primarily targets preprocessing leakage within resampling, while grouped and time-aware resampling options help reduce leakage that arises from dependence structures and temporal ordering.

Failure-mode demonstration: impact of improper batch correction

A central motivation for guarded resampling is that preprocessing leakage can arise even in workflows that otherwise appear methodologically careful. In multi-site or multi-center studies, a particularly common failure mode is improper batch correction, where site-specific normalization is performed globally prior to resampling. Although often intended to reduce heterogeneity, such preprocessing can incorporate information from held-out groups and inflate apparent performance estimates (Luo et al., 2010).

To demonstrate this failure mode and to illustrate the consequences of global preprocessing under grouped validation, we conducted a Monte Carlo simulation contrasting improper global batch correction with fold-specific preprocessing embedded in a resampling workflow. This simulation is presented as *external experimental evidence in the paper*, not as a built-in demonstration performed automatically by the package. The extent to which fastml's guarded resampling path is used depends on internal execution logic and the configuration of the modeling call. The complete reproducibility code for this simulation is provided in the supplementary file `failure_mode_simulation.R`.

Simulation setting.

We simulate a binary classification problem with a latent signal shared across sites and a strong site-specific batch effect in the observed predictor. For observation i belonging to site $s(i)$, we generate a latent signal $z_i \sim N(0,1)$, a site offset $b_s \sim N(5,5)$, and an observed predictor $x_i = z_i + b_s(i)$. The outcome depends only on the latent signal, not on site membership: $p_i = logit^{-1}(2z_i)$, and $y_i \sim \text{Bernoulli}(p_i)$. Evaluation uses grouped cross-validation, holding out one entire site per fold (leave-one-site-out), and the simulation is repeated over multiple independent runs to assess stability.

We compare two workflows: - Leaky workflow (global site-wise standardization before resampling). The predictor is standardized within each site using the full dataset before folds are created. This induces leakage because observations that later appear in assessment folds contribute to the site-specific scaling parameters applied in training folds, partially aligning held-out site distributions with the training data and inflating AUC. - Guarded workflow (fold-specific preprocessing inside resampling via fastml). Preprocessing is defined in a recipe and is re-estimated on the analysis portion of each fold, then applied to the corresponding assessment portion. The site identifier is assigned an ID role to prevent it from being used as a predictor.

```
library(dplyr)
library(rsample)
library(recipes)
library(parsnip)
library(workflows)
library(yardstick)
library(purrr)
library(fastml)
library(ggplot2)
library(tidyr)
library(patchwork)

set.seed(2025)

n_sims <- 100
n_sites <- 10
n_per_site <- 100

run_iteration <- function(seed) {
  set.seed(seed)

  total_n <- n_sites * n_per_site
  sites <- rep(paste0("S", 1:n_sites), each = n_per_site)

  z <- rnorm(total_n, mean = 0, sd = 1)
  site_offsets <- rnorm(n_sites, mean = 5, sd = 5)
  x_observed <- z + rep(site_offsets, each = n_per_site)

  prob <- 1 / (1 + exp(-(z * 2)))
  y <- factor(ifelse(runif(total_n) < prob, "Case", "Control"),
    levels = c("Control", "Case")
  )

  df <- tibble(
    site = factor(sites),
    outcome = y,
    x = x_observed
)

  # Leaky workflow:
  # Global site-wise standardization before resampling
  df_leaky <- df %>%
    group_by(site) %>%
    mutate(x_scaled = as.numeric(scale(x))) %>%
    ungroup()
```

```
  folds_leaky <- group_vfold_cv(
    df_leaky,
    group = "site",
    v = n_sites
  )

  auc_leaky <- map_dbl(folds_leaky$splits, function(spl) {
    model <- rand_forest(trees = 50) %>%
      set_mode("classification") %>%
      set_engine("ranger") %>%
      fit(outcome ~ x_scaled, data = analysis(spl))

    predict(model, assessment(spl), type = "prob") %>%
      bind_cols(assessment(spl)) %>%
      roc_auc(truth = outcome, .pred_Case,
              event_level = "second") %>%
      pull(.estimate)
  })

  grouped_folds <- rsample::group_vfold_cv(
    df, group = site, v = n_sites
  )

  # Guarded workflow:
  # Fold-specific preprocessing (fastml)
  guarded_model <- fastml(
    data = df,
    label = "outcome",
    algorithms = "rand_forest",
    resamples = grouped_folds,
    metric = "roc_auc",
    event_class = "second",
    recipe = recipe(outcome ~ x + site, data = df) %>%
      update_role(site, new_role = "id") %>%
      step_normalize(x),
    tune_params = list(
      rand_forest = list(ranger = list(trees = 50))
    )
  )
  tibble(
    Leaky_AUC = mean(auc_leaky),
    Guarded_AUC = guarded_model$performance$rand_forest$ranger %>%
      filter(.metric == "roc_auc") %>%
      pull(.estimate)
  )
}
```

In this simulation, `tune_params` is used to fix the number of trees at 50 rather than to define a tuning grid; when a single value is supplied for a parameter, fastml treats it as a fixed setting.

We repeated the simulation for `n_sims` independent runs. In each run, we computed a leaky AUC (site-wise scaling performed globally before grouped CV) and a guarded AUC (fold-local preprocessing inside grouped resampling). We summarize the distribution of AUC values across runs and quantify inflation using the paired difference (Leaky minus Guarded) across simulation runs.

```
simulation_results <- map_dfr(1:n_sims, run_iteration)
```

Summary statistics across iterations quantify the inflation under improper batch correction.

```
summary_stats <- simulation_results %>%
  summarise(
    Mean_Leaky = mean(Leaky_AUC),
    SD_Leaky = sd(Leaky_AUC),
    Mean_Guarded = mean(Guarded_AUC),
    SD_Guarded = sd(Guarded_AUC)
  )

print(summary_stats)
```

The results of this simulation are reported in Section 2.3.1.

2.2 Package interface

The central entry point is `fastml()`, a single function that coordinates data splitting, preprocessing, model fitting, evaluation, and (optionally) hyperparameter tuning. Its arguments are grouped by purpose:

- **Data**: `data` (unsplit), or `train_data`/`test_data` (pre-split); `label` (outcome column); `test_size` (holdout fraction, default 0.2).
- **Algorithms**: `algorithms` (character vector or `"all"`); `algorithm_engines` (optional engine overrides); `task` (`"auto"`, `"classification"`, `"regression"`, or `"survival"`).
- **Preprocessing**: `recipe` (user-supplied recipes object), `impute_method`, `encode_categoricals`, `scaling_methods`, `balance_method`.
- **Resampling**: `resampling_method` (e.g., `"cv"`, `"repeatedcv"`, `"boot"`, `"grouped_cv"`, `"blocked_cv"`, `"rolling_origin"`, `"nested_cv"`, `"validation_split"`, `"none"`); `folds`, `repeats`, `group_cols`, `block_col`, `resamples` (user-supplied rsample object).
- **Tuning**: `tune_params` (named list of grids per algorithm–engine pair); `tuning_strategy` (`"grid"` or `"bayes"`); `tuning_complexity` (`"quick"`, `"balanced"`, `"thorough"`, `"exhaustive"`); `use_default_tuning`.
- **Evaluation**: `metric` (primary metric); `event_class`; `class_threshold`; `bootstrap_ci` (default `TRUE`); `bootstrap_samples`.
- **Execution**: `n_cores` (parallel workers); `seed`; `verbose`; `audit_mode`.

Supported algorithms. fastml exposes the following algorithm families, each dispatched to one or more engines:

- *Classification* (15 families): `logistic_reg`, `multinom_reg`, `decision_tree`, `C5_rules`, `rand_forest`, `xgboost`, `lightgbm`, `svm_linear`, `svm_rbf`, `nearest_neighbor`, `naive_Bayes`, `mlp`, `discrim_linear`, `discrim_quad`, `bag_tree`.
- *Regression* (14 families): `linear_reg`, `ridge_reg`, `lasso_reg`, `elastic_net`, `decision_tree`, `rand_forest`, `xgboost`, `lightgbm`, `svm_linear`, `svm_rbf`, `nearest_neighbor`, `mlp`, `pls`, `bayes_glm`.
- *Survival* (12 families): `cox_ph`, `penalized_cox`, `stratified_cox`, `time_varying_cox`, `survreg`, `parametric_surv`, `royston_parmar`, `piecewise_exp`, `rand_forest`, `rand_forest_survival`, `xgboost`, `xgboost_aft`. (Note: `rand_forest` and `rand_forest_survival` map to different engines — ranger and aorsf, respectively.)

Resampling methods. Nine strategies are supported: `cv` (k-fold cross-validation, default for classification/regression), `repeatedcv`, `boot` (bootstrap), `grouped_cv` (requires `group_cols`), `blocked_cv` (requires `block_col`), `rolling_origin` (requires `initial_window`

and `assess_window`), `nested_cv` (requires `outer_folds`), `validation_split`, and `none` (default for survival).

Return value and key methods. `fastml()` returns an S3 object of class `fastml` that aggregates fitted models, the preprocessing specification, evaluation outputs, and (optionally) the data partitions (see Table 1 for details). The object supports the following methods:

- `predict.fastml(object, newdata)` — applies the stored preprocessor and fitted model to new data; supports `type` options including `"class"`, `"prob"`, `"numeric"`, and `"survival"`.
- `summary.fastml(object)` — prints performance metrics, tuned parameters, and (for classification) confusion matrices.
- `plot.fastml(object)` — generates diagnostic visualizations including metric bar charts, ROC curves, calibration plots, residual diagnostics, and learning curves.
- `save_fastml()` / `load_model()` — serialization and deserialization of fastml objects via `saveRDS`/`readRDS`.

Minimal workflow example. The simplest valid call requires only data and a label:

```
library(fastml)
fm <- fastml(data = iris, label = "Species")
summary(fm)
predict(fm, newdata = new_obs)
```

Explanation and exploration utilities. `fastexplain(object, method)` provides post hoc model explanations via external packages, supporting 10 methods including DALEX-based variable importance and partial dependence, LIME, ICE, ALE, surrogate trees, interaction analysis, breakdown contributions, counterfactual explanations, fairness diagnostics, and modelStudio dashboards. `fastexplore(data)` provides pre-modeling exploratory diagnostics (variable summaries, missingness patterns, distributions, correlations, and visualizations) that remain architecturally separate from the training pipeline (Section 2.2.4).

Parallel execution. When `n_cores > 1`, fastml registers a **doFuture** backend with `future::multisession` workers. Parallel plans are automatically restored on function exit, and seed handling is configured through `future.seed` to support deterministic parallel RNG streams. Bootstrap confidence intervals (`bootstrap_ci = TRUE`, default) are computed over `bootstrap_samples` (default 500) draws from the evaluation predictions. The package passes CRAN checks and includes a test suite covering core workflows; documentation is available through standard R help pages and a package vignette.

As a rough guide, Case Study B (5-fold CV with three algorithms and tuning grids on N $\approx$ 8,500) completes in approximately 1.6 minutes on a single core (Intel Core i5-11400 @ 2.60 GHz) using R 4.5. Parallel execution with `n_cores = 4` reduces this to approximately 0.9 minutes. These timings are indicative and will vary with hardware, algorithm complexity, and dataset size.

2.3 Native survival analysis implementation

Survival support: learners, outcomes, and resampling

fastml supports a set of survival learners that require explicit handling of censoring and, for some engines, specialized prediction objects. In the current release, **fastml** includes Cox family models: Cox proportional hazards (`cox_ph`), penalized Cox regression (`penalized_cox`), stratified Cox regression (`stratified_cox`), and time varying Cox regression (`time_varying_cox`). It also includes parametric survival models: Accelerated Failure Time regression (`survreg`), generic parametric survival (`parametric_surv`), Royston Parmar flexible parametric survival (`royston_parmar`), and a custom piecewise exponential model (`piecewise_exp`). Machine learning survival learners include oblique random survival forest survival (`rand_forest_survival`) and XGBoost accelerated failure time (`xgboost_aft`).

These methods are trained through a shared interface but are dispatched to different fitting backends depending on method and configuration.

Outcome representation. **fastml** represents survival outcomes internally using a survival response constructed from follow-up time and event status (i.e., a `Surv(time, status)` object) for standard right-censored data (Therneau and Grambsch, 2000; Therneau, 2024). For models that require alternative encodings, **fastml** constructs the appropriate target representation explicitly. In particular, the `xgboost_aft` backend uses lower and upper interval bounds for each observation (exact bounds for events; one-sided bounds for right-censoring) rather than a single response value, aligning with the AFT objective function (Barnwal et al., 2020). Basic validation checks ensure that the required outcome fields are present and can be coerced into the expected form, but **fastml** does not modify censoring patterns (e.g., it does not rebalance events or alter follow-up times).

Prediction conventions. Survival engines differ in what they naturally return (risk scores, linear predictors, survival curves, time quantiles, or distributional parameters). **fastml** therefore standardizes predictions only to the extent needed for evaluation: Cox-type learners provide a risk score/linear predictor suitable for concordance-style evaluation (Harrell et al., 1982), parametric learners provide model-based predictions on the scale implied by the fitted distribution, and `xgboost_aft` provides predictions aligned with its AFT objective (typically on the log-time scale, with evaluation defined accordingly). When metrics require time-indexed quantities, evaluation uses explicitly supplied time points rather than assuming that every engine can natively return a full survival curve.

Fitting under resampling and tuning. **fastml** uses conditional dispatch to choose between workflow-based fitting (via tidymodels abstractions) and direct engine calls. When resampling and/or hyperparameter tuning is requested and a method is supported through a resampling-aware workflow, **fastml** fits preprocessing and model training within each split so that fold-local preprocessing is preserved. Methods that are not executed through the workflow-based resampling path are fit using their native interfaces and evaluated under the corresponding train/test or resampling configuration supported by the training pipeline. Practically, this means that "resampling-aware" fitting is method- and configuration-dependent, and survival resampling is more constrained than for classification/regression tasks.

A minimal survival call illustrates the unified interface. The example below uses `cox_ph`, which depends only on the **survival** package (always available). Additional survival algorithms such as `penalized_cox` and `rand_forest_survival` are available when the optional **censored** package is installed.

```
library(fastml)
library(survival)
library(censored)

lung_df <- lung[complete.cases(lung), ]
lung_df$status <- lung_df$status - 1  # recode from 1/2 to 0/1

fm_surv <- fastml(
  data       = lung_df,
  label      = c("time", "status"),
  algorithms = "cox_ph",
  metric     = "c_index",
  seed       = 2025
)

summary(fm_surv, type = "metrics")

#>
#> ===== fastml Model Summary =====
#> Task: survival
```

```
#> Number of Models Trained: 1
#> Best Model(s): cox_ph (survival) (c_index: 0.6600000)
#>
#> Performance Metrics (Sorted by c_index):
#>
#> ---------------------------------------------------------------------------------------------
#> Model   Engine   Harrell C-index  Uno's C-index  Integrated Brier Score  RMST diff (t<=655)  Brier(t=
#> ---------------------------------------------------------------------------------------------
#> cox_ph* survival 0.660        0.702       0.208             82.158          0.233      0.238
#> ---------------------------------------------------------------------------------------------
#> (*Best model)
```

Explicit non-support and constraints. The survival interface is intentionally scoped. Not all survival data structures and evaluation designs are supported in every backend. For example, the `xgboost_aft` path supports right-censoring through interval bounds but does not accept counting-process (start-stop) outcomes. More generally, some combinations of survival method, resampling design, and tuning configuration are restricted by the training pipeline (and may be rejected with an error) rather than silently coerced. These constraints are treated as part of the package's design: **fastml** aims to make supported survival workflows easy to execute correctly, while failing fast when a requested configuration is not implemented or would be ambiguous.

XGBoost accelerated failure time model with interval bounds

One of the native survival implementations in **fastml** is the Accelerated Failure Time (AFT) model using XGBoost's survival objective. This path does not rely on a formula interface. Instead, **fastml** constructs the necessary inputs explicitly and calls the lower-level XGBoost API to ensure correct handling of censoring. The AFT model assumes a parametric form for log-survival time, modeling

$$\log(T_i) = \eta_i + \varepsilon_i, \tag{1}$$

where η_i is a predictor-dependent location parameter and ε_i follows a specified error distribution (e.g., Normal, Logistic, or Extreme Value) (Kalbfleisch and Prentice, 2002). To represent censoring correctly, XGBoost requires interval-censored targets rather than single point estimates.

For each observation with observed time t_i and event indicator δ_i, **fastml** computes

$$\ell_i = \log(t_i), \tag{2}$$

and defines the target interval as $[\ell_i, u_i)$ where $u_i = \ell_i$ for uncensored observations ($\delta_i = 1$) and $u_i = \infty$ for right-censored observations ($\delta_i = 0$). These lower and upper bounds are supplied to XGBoost by setting `label_lower_bound` and `label_upper_bound` in the `xgb.DMatrix`. By doing so, **fastml** ensures that right-censored observations contribute a one-sided constraint to the loss function, indicating that the event time exceeds the observed value without assuming an exact failure time. This avoids treating censored times as exact regression targets (Barnwal et al., 2020).

The implementation operates directly on log-time and interval bounds and does not support start-stop (counting process) survival data; such outcomes are explicitly rejected for the XGBoost AFT path. **fastml**'s role here is to translate standard right-censored survival data into the interval-censored representation required by XGBoost's AFT objective, without altering the underlying censoring information or relying on inappropriate regression abstractions.

Custom piecewise exponential distribution

In addition to standard parametric survival models, **fastml** implements a custom piecewise exponential distribution integrated with the **flexsurv** framework (Jackson, 2016). The goal is to allow flexible baseline hazard shapes while retaining a parametric likelihood and interpretable parameters.

The piecewise exponential model assumes that the hazard function is constant within predefined time intervals (Friedman, 1982). Let $0 < c_1 < c_2 < \cdots < c_K$ denote user-supplied cutpoints. Within each interval $(c_{k-1}, c_k]$, the hazard is assumed to be

$$h(t) = \lambda_k, \tag{3}$$

where each $\lambda_k > 0$ is an interval-specific hazard rate. **fastml** implements this model by defining the hazard, cumulative hazard, density, distribution, and quantile functions algorithmically in a custom distribution family compatible with **flexsurv**.

User-provided breakpoints are normalized to positive, finite values; **fastml** does not automatically append an infinite upper bound, nor does it enforce a specific number of intervals. Internally, the custom distribution parameterizes hazards using a baseline log-rate (`log_rate`) and a sequence of log hazard ratios (`log_ratio_1`, `log_ratio_2`, ...), which are then converted into interval-specific rates λ_k. This parameterization is used to ensure positivity and stable estimation while still yielding hazard rates that correspond to piecewise-constant intervals.

The cumulative hazard at time t is computed as the sum of hazards over all completed intervals plus the partial contribution of the interval containing t. The survival function is then

$$S(t) = \exp(-H(t)), \tag{4}$$

and the density follows directly from the hazard and survival functions. These expressions correspond to the standard piecewise constant hazard formulation, while the implementation itself evaluates them procedurally.

Model fitting is performed via `flexsurv::flexsurvreg()`, with **fastml** passing the custom distribution definition and the normalized cutpoints through its internal fitting utilities. This allows the piecewise exponential model to be estimated using maximum likelihood in the same manner as built-in **flexsurv** distributions, while offering greater flexibility than single-parameter parametric forms such as the exponential or Weibull (Lawless, 2003). Each estimated interval hazard rate retains an interpretation as the event rate within a specified time band, making the model useful when hazard rates are expected to vary across follow-up time.

2.4 Software architecture and security

Beyond statistical validity, an AutoML framework must be robust as software. In practice, this means returning results in a form that is easy to inspect, serialize, and reuse, while also reducing the chance that extensibility mechanisms introduce hidden dependencies or unintended side effects (Wilson et al., 2014). In **fastml**, these goals are addressed through (i) a structured S3 object (Wickham, 2019) that centralizes fitted models, preprocessing, evaluation outputs, and (optionally) the data partitions used during training and evaluation, and (ii) a set of security and audit utilities that can surface risky patterns in user-supplied recipes and hooks.

The audit and validation utilities are designed to reduce hidden dependencies and common misuse, such as reliance on global environment variables which harms reproducibility (Peng, 2011), but they do not provide data-security guarantees; confidentiality depends on what the user chooses to retain and how objects are stored and shared.

Table 1: Core components of a 'fastml' object and associated guarantees.

Slot	Description	F
`models`	Fitted model objects, indexed by algorithm and engine	M
`preprocessor`	Stored preprocessing object	T
`predictions`	Stored predictions from evaluation	F
`performance`	Aggregated performance metrics	M
`metric`	Primary evaluation metric	D
`best_model_name`	Identifier of the default reference model	A
`resampling_plan`	Resampling specification (e.g., CV folds, repeats)	E
`raw_train_data`, `raw_test_data`	Data partitions used for fitting/evaluation (when retained)	E
`processed_train_data`, `processed_test_data`	Preprocessed partitions (when retained)	S
`audit`	Audit log entries and a flagged indicator	W

The object model: a self-contained result object

Many R workflows store fitted models, preprocessing artifacts, predictions, and evaluation results as separate objects, or recreate them on demand. **fastml** instead returns a single S3 object that aggregates the key artifacts produced during training and evaluation. The object is intended to be sufficient for prediction and plotting in a stable/compatible R environment, given the stored fitted model objects and preprocessing specification. However, portability across machines or R installations is not guaranteed for all engines or workflows, because some fitted objects may contain external pointers or compiled components (Eddelbuettel, 2013), serialization behavior can be version-dependent, and prediction may depend on the availability and behavior of upstream packages and recipe steps. Reproducibility also depends on package versions, RNG state, and platform-specific numerical differences (Stodden et al., 2014). In particular, the **fastml** object: - stores fitted model objects, organized by algorithm and engine, - stores the preprocessing object used for prediction, - stores evaluation artifacts used for reporting (notably holdout/test predictions and performance summaries), - and may retain the raw and processed training/test partitions used for fitting and evaluation, enabling inspection and post hoc validation. This improves portability and auditability, but also means the object may carry data and is therefore not "data-free".

The object represents either a resampling-based workflow or a fixed holdout workflow. When resampling is requested and executed, the object records the resampling plan and summary performance outputs associated with that plan. When a single train-test split is used, resampling-related fields are absent or `NULL`, and reported metrics refer to the holdout/test evaluation.

Note: This table lists the principal user-facing components. The full object contains additional fields (e.g., `task`, `label`, `event_class`, `engine_names`, `resampling_results`, `metric_bootstrap`) that support internal dispatch and are accessible for advanced use.

Table 1 summarizes common components of a **fastml** object and the practical guarantees they provide. This object contract is used throughout the package. Plotting methods are intended to operate on stored predictions and performance summaries rather than recomputing from scratch. Similarly, `predict.fastml` applies the stored preprocessor and stored fitted model(s), yielding consistent predictions within a stable R environment. The object is designed to be serializable (e.g., via `saveRDS`), enabling reuse in downstream workflows.

Security and audit utilities

AutoML frameworks often support extensibility through custom recipe steps, custom metrics, or user hooks. This extensibility introduces two practical risks: (i) hidden dependencies (for example, reliance on `.GlobalEnv` or objects defined outside the modeling call), which undermines reproducibility and auditability, and (ii) side effects (for example, reading from

files, writing outputs to files, or modifying global objects), which can make automated pipelines fragile.

fastml addresses these risks through two distinct mechanisms that serve different purposes: (a) recipe validation, which is preventive but intentionally narrow, and (b) audit-mode instrumentation, which is broader but observational and incomplete. Neither mechanism should be interpreted as a comprehensive security boundary inside R. Users can bypass many checks via qualified namespace calls (e.g., `base::`), alternative I/O functions and connections, `system`, non-local assignment, `getFromNamespace`, or side effects in compiled code.

Recipe validation (preventive, narrow). **fastml** includes a recipe-scanning guard (implemented in `R/security_guards.R`) that focuses on a small class of high-impact issues that commonly produce hidden dependencies or fold-external data injection. In particular, it scans recipe steps for patterns suggesting reliance on external environments or global objects (e.g., `.GlobalEnv`, parent-frame access, or related environment inheritance patterns) and flags steps that appear to embed external data objects (for example, steps inheriting from `data.frame`). Recipes flagged as depending on external environments or embedded data objects are rejected before model training begins, with an informative message. This guard is not a full static analyzer and it does not attempt to detect file reads/writes or arbitrary unsafe operations inside recipe code.

Audit-mode instrumentation (observational, broader but incomplete). When `audit_mode = TRUE`, **fastml** can run parts of the pipeline under an instrumented environment that records and flags selected risk patterns during execution. This includes (i) flagging common forms of global-environment dependence under the instrumented execution path (for example, by detecting attempted `.GlobalEnv` access and monitoring calls to functions such as `assign`, `rm`, and `get` when they target the global workspace), and (ii) flagging or recording usage of common functions that read from files or write outputs to files (for example, wrappers around frequently used read/write helpers). **fastml** may also perform lightweight symbol/name scans of function bodies (e.g., via `all.names`) to flag obvious `.GlobalEnv` and I/O-related patterns. These checks are best interpreted as audit signals under the instrumented execution path rather than as proof of safety or guarantees of prevention.

Integration with the training pipeline. Recipe validation is invoked before model training to prevent a narrow class of unsafe recipe definitions. Audit logging is invoked when `audit_mode = TRUE` to emit warnings and record flags. Helper utilities for instrumented execution of arbitrary user hooks may exist, but they are not necessarily used by the default training flow; the audit mechanism should therefore be described as optional and configuration-dependent.

If contrasted with OS-level solutions (e.g., **RAppArmor** (Ooms, 2013)), the appropriate framing is that OS-level tools can enforce stronger process constraints, whereas **fastml** operates within the R runtime and focuses on targeted prevention (recipe validation) plus optional observability (`audit_mode`) for common reproducibility risks.

Exploratory diagnostics and architectural isolation

Before initiating automated model training, workflows commonly include an exploratory assessment of variable types, missingness patterns, distributions, and data-quality issues. In fastml, this is supported by an optional diagnostic utility, `fastexplore`, designed to remain separate from the automated resampling and training core.

First, `fastexplore` is not called internally by `fastml` and does not participate in model fitting, recipe estimation inside resampling, or evaluation. Second, its default behavior is low side-effect: it returns structured summaries (tables and plots) without writing to disk. Disk output is explicitly opt-in through arguments such as `save_results` or `render_report`.

In practice, `fastexplore` produces variable-level summaries (types, unique counts, descriptive statistics), missingness diagnostics (per-variable rates, missing-data pattern visualizations including UpSet plots), distribution checks (normality tests, skewness detection), correlation analysis (pairwise matrices with configurable thresholds for flagging high collinearity), and grouped visualizations (histograms, boxplots, bar charts, heatmaps, and scatterplots). These outputs are intended to inform modeling configuration decisions (for example, whether to impute, transform, or remove features), but they do not automatically alter the training pipeline. Any actions based on diagnostics must be encoded explicitly by the user in the subsequent modeling call.

2.5 Comparison with existing frameworks

The R ecosystem offers several mature frameworks for machine learning. To contextualize **fastml**, we compare it against four widely used alternatives: **caret** (Kuhn, 2008) (legacy unified interface), **tidymodels** (Kuhn and Wickham, 2020) (modular workflow system), **mlr3** (Lang et al., 2019) (object-oriented framework with a strong benchmarking culture), and **h2o** (Fryda et al., 2020) (an AutoML platform with an internal training/evaluation engine). The comparison below combines (i) architectural differences relevant to leakage risk, survival modeling, and workflow ergonomics, and (ii) external benchmarks conducted for this paper. Implementation details described as properties of **fastml** refer to its current codebase; statements about other frameworks reflect their documented behavior and typical usage patterns rather than properties established by **fastml**'s code.

Feature comparison matrix

Frameworks differ in where they sit on the flexibility-to-automation spectrum and in how explicitly users must assemble preprocessing, resampling, model fitting, and evaluation. **caret** provides a largely unified training interface; **tidymodels** emphasizes explicit composition (recipes + model specification + workflow + rsample); **mlr3** provides an object-oriented task/learner design with strong support for benchmarking; and **h2o** provides an integrated AutoML system with its own training and evaluation engine. **fastml** adopts a single-call façade that orchestrates preprocessing, fitting, and evaluation while exposing a guarded resampling path and diagnostics intended to reduce common workflow errors.

A key comparison dimension is how preprocessing is coupled to evaluation. **tidymodels** and **mlr3** are flexible enough to express both leakage-safe and leakage-prone workflows; leakage-resistant evaluation is obtained when preprocessing is estimated on the analysis portion of each resample and then applied to the corresponding assessment data. **fastml**'s guarded resampling path is designed to re-estimate preprocessing within each resample when that execution mode is used, reducing the risk of preprocessing leakage under supported configurations. This should be interpreted as risk reduction rather than a universal guarantee (Section 2.1; Section 2.2.1).

Survival-model support also differs across ecosystems in both breadth and evaluation tooling. **caret**, **tidymodels**, and **mlr3** offer survival modeling through different abstractions and extension packages (e.g., **censored** package (Hvitfeldt and Frick, 2025)), while **fastml** exposes a set of survival methods through a unified interface and dispatches between parsnip-based workflows and native engines depending on whether resampling/tuning is requested (Section 2.2.3). The practical set of methods available to any wrapper framework ultimately depends on engine availability and on which internal execution path is invoked.

Finally, extensibility and "safety" features should be interpreted precisely. **fastml** includes preventive recipe validation plus optional audit-mode instrumentation aimed at surfacing common sources of hidden dependencies and side effects (Section 2.2.4). These utilities improve auditability under an instrumented execution path, but they do not constitute a security boundary inside R. Table 2 highlights differences in design philosophy, safety enforcement, and domain-specific support.

Table 2: Architectural and functional comparison of major R machine learning frameworks.

Feature	caret	tidymod
Interface philosophy	Unified wrapper; function-based syntax such as `train`.	Modular
Preprocessing and leakage control	Manual; users must prevent leakage explicitly.	Optional
Survival analysis support	Basic support, primarily Cox proportional hazards.	Expandi
Extensibility and safety	Standard execution inside the R session.	Standarc
Parallel execution	Via **doParallel** or similar foreach backends.	Via **futu**
Model explainability	Limited built-in support; variable importance for some models.	Via **DAL**

3 Results

3.1 Simulation results

Across 100 Monte Carlo runs, the leaky workflow achieved a mean ROC AUC of 0.809 (SD: 0.018), whereas the guarded workflow achieved a mean ROC AUC of 0.651 (SD: 0.046).

```
diffs <- simulation_results$Leaky_AUC - simulation_results$Guarded_AUC
mean_diff <- mean(diffs)
se_diff <- sd(diffs) / sqrt(n_sims)
ci <- mean_diff + c(-1, 1) * qt(0.975, df = n_sims - 1) * se_diff

mean_diff
ci
```

The paired inflation, defined as Leaky AUC - Guarded AUC within each run, was 0.158 on average, with a 95% t-based confidence interval of $[0.149, 0.167]$.

These results indicate a large and consistent upward bias in apparent discrimination when site-wise normalization is performed globally prior to grouped validation. The guarded workflow substantially reduces this bias by re-estimating preprocessing within each resample and applying fold-trained transformations only to the corresponding held-out site.

Figure 2A shows the distribution of ROC AUC values across runs for the leaky and guarded workflows. Figure 2B shows the paired within-run change in AUC, highlighting the systematic drop in performance when preprocessing is moved inside the resampling loop.

```
p1 <- simulation_results %>%
  pivot_longer(cols = c(Leaky_AUC, Guarded_AUC)) %>%
  ggplot(aes(x = name, y = value, fill = name)) +
  geom_violin(alpha = 0.3, trim = FALSE) +
  geom_boxplot(width = 0.2, alpha = 0.8) +
  scale_fill_manual(values = c("#99CC99", "#FF9999")) +
  labs(title = "A. AUC Distribution", y = "ROC AUC", x = NULL) +
  theme_minimal() + theme(legend.position = "none")

p2 <- ggplot(simulation_results) +
  geom_segment(aes(
    x = "Leaky",
    xend = "Guarded",
    y = Leaky_AUC,
    yend = Guarded_AUC
  ),
  alpha = 0.3) +
  geom_point(aes(x = "Leaky", y = Leaky_AUC), color = "#FF9999") +
```

```
  geom_point(aes(x = "Guarded", y = Guarded_AUC), color = "#99CC99") +
  labs(title = "B. Paired Drop", y = "ROC AUC", x = NULL) +
  theme_minimal()

p1 + p2
```

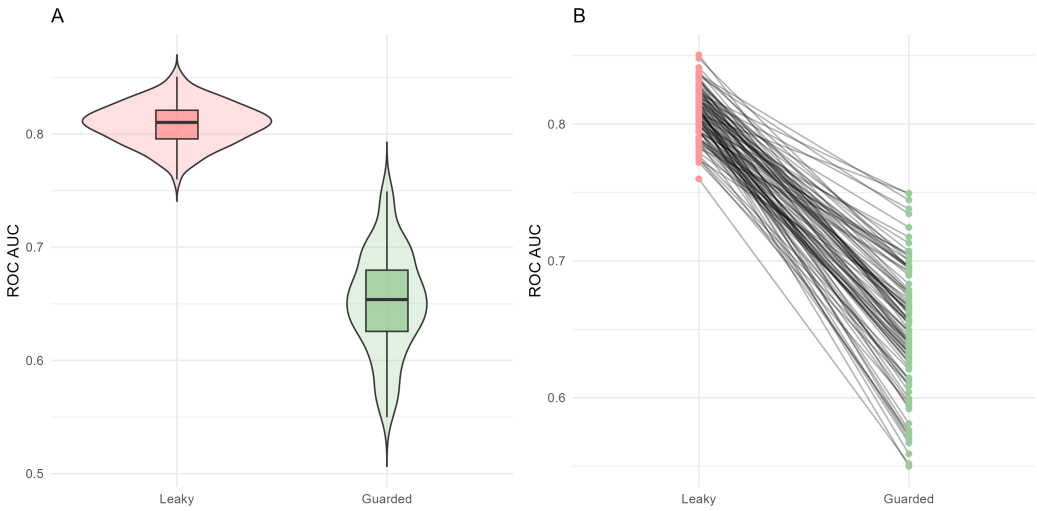


Figure 2: Preprocessing placement under grouped cross-validation: (A) ROC AUC distributions for leaky versus guarded workflows; (B) paired ROC AUC showing inflation with global preprocessing.

This experiment demonstrates a concrete and practically relevant failure mode: site-wise normalization performed prior to grouped validation can substantially inflate apparent discrimination, even without access to outcome labels. The paired reductions in ROC AUC show that re-estimating preprocessing within each fold is necessary for valid evaluation when group-level structure is present. In fastml, guarded resampling reduces this class of error by fitting preprocessing and models within resamples when the guarded execution path is used (Section 2.1).

3.2 Benchmarks

Benchmark A: usability and code efficiency

To quantify differences in user-facing orchestration burden, we implemented a standard clinical classification task using the Pima Indians Diabetes dataset (Smith et al., 1988) (N = 768; 8 predictors; missingness in physiologically related variables) in the **mlbench** package (Leisch and Dimitriadou, 2024). This benchmark is not intended to claim that **fastml** and **tidymodels** are definitionally equivalent in preprocessing or modeling defaults. Instead, it compares the amount of explicit pipeline construction typically required to obtain a valid holdout evaluation when preprocessing is fitted on the training partition and then applied to the test partition.

Common setup (data splitting). We define a single stratified train-test split and supply the same partitions to both approaches so that evaluation is performed on identical holdout data.

tidymodels implementation (explicit pipeline). A tidymodels workflow typically requires the user to explicitly define preprocessing, assemble a workflow, fit the model, generate predictions, and compute metrics. Because the Pima dataset is entirely numeric, the `step_novel`, `step_unknown`, and `step_dummy` calls below are no-ops for this particular dataset; a minimal, dataset-specific recipe could omit them. We retain them here to illustrate the typical boilerplate a practitioner writes when building a reusable template that must

also handle nominal predictors. The code-volume comparison should therefore be read as reflecting a common template-driven workflow rather than the minimal possible recipe for this dataset.

```
library(tidymodels)

rec <- recipe(diabetes ~ ., data = train_data) %>%
  step_impute_median(all_numeric_predictors()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors()) %>%
  step_center(all_numeric_predictors()) %>%
  step_scale(all_numeric_predictors())

log_spec <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")

wf <- workflow() %>%
  add_recipe(rec) %>%
  add_model(log_spec)

final_fit <- fit(wf, data = train_data)

preds <- predict(final_fit, new_data = test_data) %>%
  bind_cols(predict(final_fit, new_data = test_data, type = "prob")) %>%
  bind_cols(test_data)

multi_metric <- metric_set(
  accuracy, kap, f_meas, precision,
  sens, spec, roc_auc
)

metrics_tm <- multi_metric(
  preds,
  truth = diabetes,
  estimate = .pred_class,
  .pred_pos,
  event_level = "second"
)

print(metrics_tm)

#> # A tibble: 7 x 3
#>   .metric   .estimator .estimate
#>   <chr>     <chr>          <dbl>
#> 1 accuracy  binary         0.776
#> 2 kap       binary         0.491
#> 3 f_meas    binary         0.656
#> 4 precision binary         0.707
#> 5 sens      binary         0.612
#> 6 spec      binary         0.864
#> 7 roc_auc   binary         0.853
```

fastml implementation (single-call interface). fastml accepts pre-split data and orchestrates preprocessing, model fitting, prediction, and evaluation within a single call. The

specific preprocessing applied depends on the arguments provided and package defaults; therefore, numerical parity with the preceding **tidymodels** example should only be claimed when preprocessing choices are explicitly matched.

```
library(fastml)

fm <- fastml(
  train_data    = train_data,
  test_data     = test_data,
  label         = "diabetes",
  algorithms    = "logistic_reg",
  impute_method = "medianImpute",
  event_class   = "second"
)

fm$performance$logistic_reg

#> $glm
#> # A tibble: 7 x 3
#>   .metric   .estimator .estimate
#>   <chr>     <chr>          <dbl>
#> 1 accuracy  binary         0.776
#> 2 kap       binary         0.491
#> 3 sens      binary         0.612
#> 4 spec      binary         0.864
#> 5 precision binary         0.707
#> 6 f_meas    binary         0.656
#> 7 roc_auc   binary         0.853
```

Because both implementations use the same model engine (`glm`), the same training/test partition, and equivalent preprocessing steps, the outputs shown above can be compared directly. When preprocessing choices are matched, the two approaches yield the same holdout metrics, confirming that **fastml** dispatches to the underlying **tidymodels** engine without altering the fitted model.

Beyond code volume, the comparison highlights a difference in where correctness effort is concentrated. In a manual **tidymodels** workflow, the user must explicitly define and correctly sequence each preprocessing step, assemble a workflow object, generate predictions, and compute metrics. Each added step creates an opportunity for omission or mis-ordering. **fastml** reduces this surface area by bundling common preprocessing, fitting, and evaluation into a single call with safe defaults. This lowers the chance of accidental leakage within the supported pipeline (Section 2.1).

Benchmark B: survival analysis across data scales

To evaluate survival modeling behavior under different data regimes, we conducted an external benchmark comparing classical baselines with fastml-exposed survival engines across datasets of substantially different sizes. This benchmark is intended to illustrate how a single interface can be used to run multiple survival model classes under a consistent evaluation plan. The benchmark is "external" in the sense that all splits, metrics, and summaries are defined by the experimental code in this paper rather than being properties enforced by **fastml** itself. The complete reproducibility code for this benchmark is provided in the supplementary file `survival_benchmark.R`.

Datasets. We used widely available survival datasets representing distinct scales: - **Small**: NCCTG Lung Cancer (`lung`, $N = 228$) (Loprinzi et al., 1994) - **Medium**: Rotterdam Breast Cancer (`rotterdam`, $N = 2,982$) (Foekens et al., 2000) - **Large**: Serum Free Light Chain (`flchain`, $N = 7,874$) (Kyle et al., 2006)

Evaluation protocol. Performance was estimated using repeated 5-fold cross-validation (3 repeats). Folds were stratified by event status to stabilize the censoring/event mix across splits, a common pragmatic choice in finite-sample survival benchmarking. Reported values summarize the distribution of fold-level concordance scores across all resamples.

Models compared. We evaluated two classical baselines implemented outside **fastml** using the same resampling splits and metric definitions: Cox proportional hazards (`survival::coxph`) and Weibull AFT (`survival::survreg`), alongside fastml-exposed survival methods (e.g., penalized Cox, random-forest survival, and XGBoost AFT) using fixed, pre-specified engine settings. To avoid confounding evaluation with model selection, hyperparameter tuning was not performed in this benchmark; any non-default parameter settings were chosen a priori and held constant within each dataset.

Concordance standardization. Because some survival engines may return risk scores whose sign convention differs from the standard (higher risk = shorter survival), raw concordance values below 0.5 are replaced by $1 - C$ to ensure a consistent directionality across all models. This "standardized" C-index always lies in $[0.5, 1]$, where 0.5 indicates no discrimination.

Uncertainty summaries. For each dataset–model combination, we report the mean concordance across resamples with the corresponding standard deviation, and 95% confidence intervals computed from the empirical quantiles of resample scores. Because cross-validation fold estimates are not independent, these intervals are descriptive summaries of resampling variability rather than formal inferential confidence intervals (Bengio and Grandvalet, 2004).

Results reporting. Table 2.3.2 reports mean C-index ± SD with 95% confidence intervals computed over the resamples (5 folds × 3 repeats = 15 evaluations). All numeric values in such a table are external experimental results from this study's benchmarking code.

\begin{table}

\caption{Mean standardized Harrell's C-index +/- SD (95% CI) across data scales from the external benchmark.}

Dataset	Cox PH (Baseline)	Weibull AFT (Baseline)	fastml (Penalized Co
Lung (N = 228)	0.631 +/- 0.064 (0.530 to 0.716)	0.633 +/- 0.062 (0.535 to 0.716)	0.641 +/- 0.066 (0.528
Rotterdam (N = 2,982)	0.690 +/- 0.014 (0.669 to 0.713)	0.690 +/- 0.014 (0.669 to 0.712)	0.695 +/- 0.015 (0.673
Flchain (N = 7,874)	0.794 +/- 0.013 (0.775 to 0.817)	0.794 +/- 0.013 (0.776 to 0.817)	0.793 +/- 0.012 (0.776

\end{table}

Any narrative interpretation (e.g., "nonlinear models outperform linear baselines in medium-scale settings") should be phrased as conclusions from this benchmark under the stated protocol, not as an implication that fastml enforces those outcomes. The main contribution to highlight here is workflow comparability: multiple survival model classes can be evaluated under a single, consistent interface and metric convention, which reduces the cost of comparing methods across regimes, provided the evaluation design (splits, preprocessing placement, and parameter constraints) is defined carefully by the experimenter.

3.3 Case studies

This section presents two applied examples that mirror common biomedical workflows: (i) comparative diagnostic modeling for binary classification, and (ii) resampling-based regression benchmarking under a fixed tuning budget. The goal is to show how fastml coordinates preprocessing, model fitting, and evaluation within a single interface while keeping key experimental choices explicit (data filtering, random seed, resampling plan, and tuning grid).

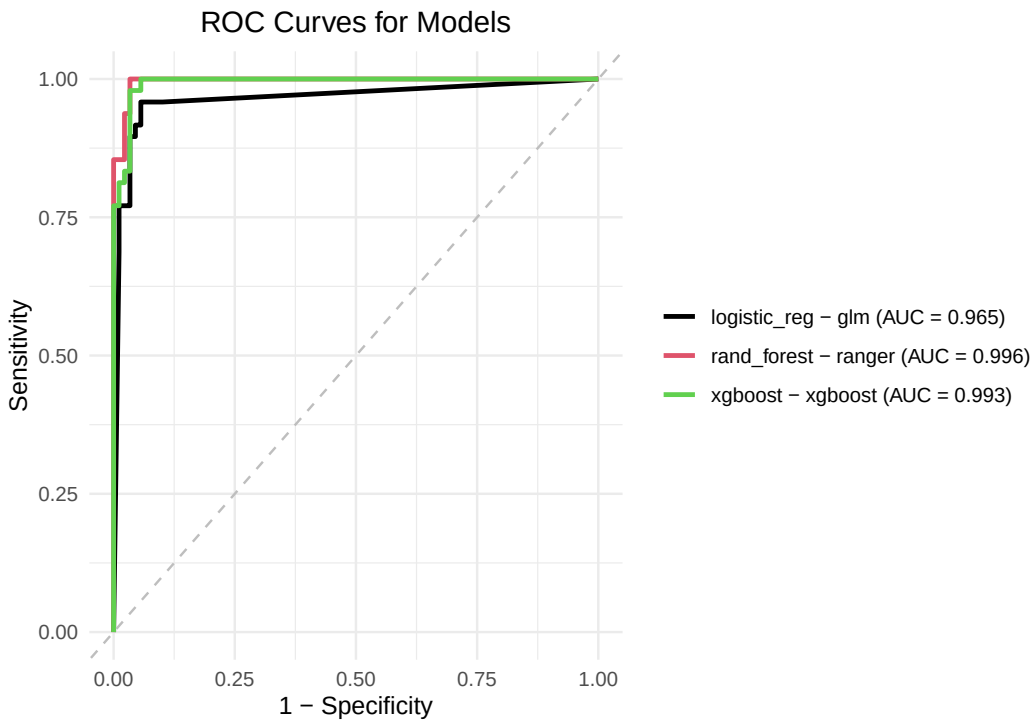
All reported metrics (e.g., AUC, RMSE) are empirical outputs of the specific code shown and therefore depend on the declared evaluation design (holdout vs resampling), tuning scope, random seed, and software environment (package versions and available engines).

Case study A: diagnostic benchmarking (breast cancer)

In diagnostic research, it is common to compare a linear baseline with one or more nonlinear learners to evaluate whether additional model flexibility improves discrimination. We illustrate this pattern using the Wisconsin Breast Cancer dataset (`mlbench::BreastCancer`, N = 569) (Street et al., 1993) and three algorithm classes: logistic regression, random forest (Breiman, 2001), and gradient boosting (Chen and Guestrin, 2016). The primary outcome is the ROC AUC on the evaluation data generated under the specified workflow.

The dataset includes an identifier column (Id), which is not a meaningful predictor and should be excluded to avoid distorted performance estimates. The code below removes Id, ensures the outcome (Class) is observed, and uses complete-case analysis for simplicity. When missingness is substantial or plausibly informative, missing-data handling should be modeled explicitly (e.g., via an imputation recipe) rather than addressed by dropping incomplete rows (Little and Rubin, 2019).

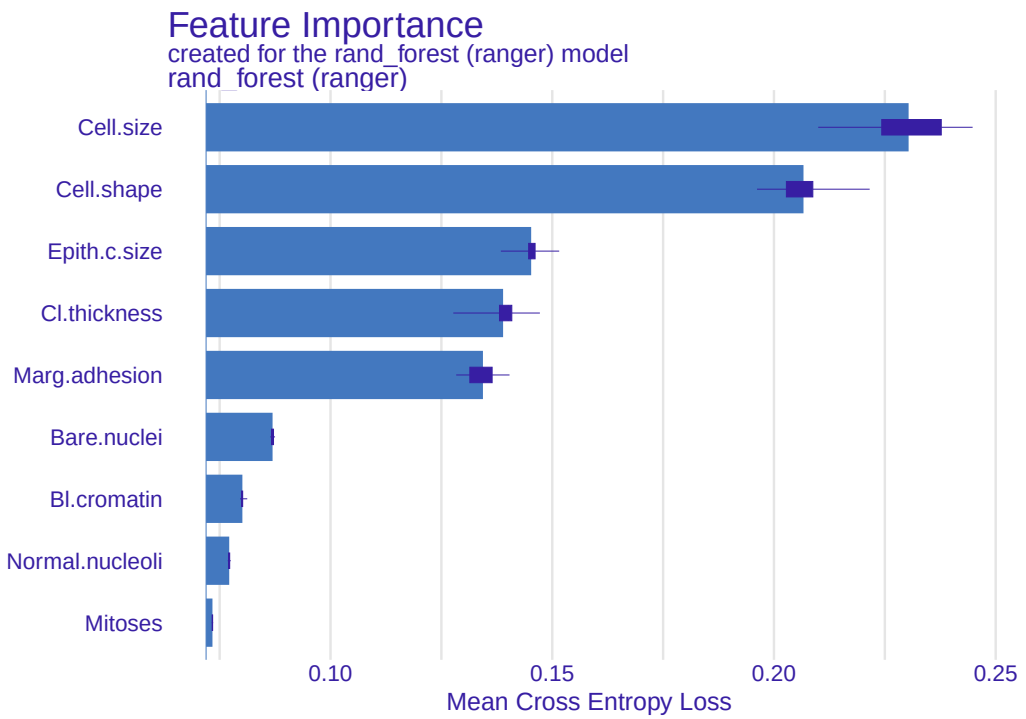
```
library(fastml)
library(mlbench)
library(dplyr)

set.seed(2025)

data("BreastCancer")

bc <- BreastCancer %>%
  select(-Id) %>%           # exclude identifier
  filter(!is.na(Class)) %>% # ensure outcome observed
  na.omit()                 # simplify this case study: complete-case analysis

fm_bc <- fastml(
  data          = bc,
  label         = "Class",
  algorithms    = c("logistic_reg", "rand_forest", "xgboost"),
  metric        = "roc_auc",
  event_class   = "second"
)

summary(fm_bc, type = "metrics")

#>
#> ===== fastml Model Summary =====
#> Task: classification
#> Number of Models Trained: 3
#> Best Model(s): rand_forest (ranger) (roc_auc: 0.9960206)
#>
#> Performance Metrics (Sorted by roc_auc):
#>
#> ---------------------------------------------------------------------------------------------
#> Model      Engine  Accuracy  F1 Score  Kappa  Precision  Sensitivity  Specificity  ROC AUC
#> ---------------------------------------------------------------------------------------------
#> rand_forest* ranger  0.971    0.960    0.937  0.923    1.000      0.955      0.996
#> xgboost     xgboost  0.964    0.948    0.920  0.939    0.958      0.966      0.993
#> logistic_reg glm    0.942    0.915    0.870  0.935    0.896      0.966      0.965
#> ---------------------------------------------------------------------------------------------
#> (*Best model)

plot(fm_bc, type = "roc")
```



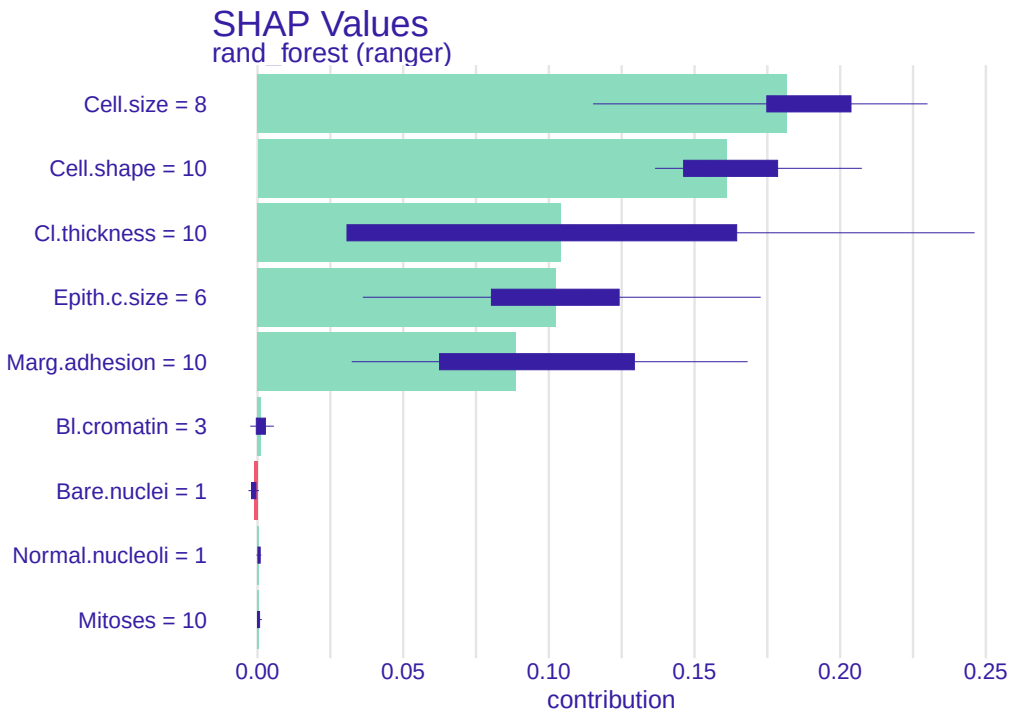
The summary output provides a compact comparison of model performance under a common preprocessing and evaluation configuration. The ROC visualization complements the metric table by showing trade-offs in sensitivity and specificity across thresholds.

Model explanation is often desirable in diagnostic applications. **fastml** provides `fastexplain` for post hoc explanations via external explainability tooling. The availability of specific explanation artifacts (e.g., permutation importance, SHAP-style outputs) depends on installed explainer packages and on model/engine compatibility (Lundberg and Lee, 2017); therefore, explanation figures should be reported only when they are produced in the paper's software environment.

```
# Optional: explanation outputs depend on installed explainers
#           and model compatibility
fastexplain(fm_bc, method = "dalex")

#>
#> === DALEX Variable Importance (with Boxplots) ===
```




```
#>
#> === DALEX Shapley Values (SHAP) ===
```



Case study B: mixed-type clinical regression (hypertension) The second example demonstrates resampling-based regression benchmarking in a larger, mixed-type clinical dataset. Using NHANES data (N ≈ 8,500 after filtering to observed outcomes) (Pruim, 2025), the objective is to predict systolic blood pressure (`BPSysAve`) from demographic, lifestyle, and laboratory variables. We compare three model families: elastic net (regularized linear model) (Zou and Hastie, 2005), random forest (bagging ensemble), and **LightGBM** (gradient boosting) (Ke et al., 2017). Hyperparameters are tuned using a predefined grid under 5-fold cross-validation.

This design reflects a practical goal: evaluate whether a linear model is sufficient, or whether nonlinear learners provide measurable gains under a fixed and transparent tuning budget. Engine availability is an external requirement (e.g., LightGBM via the **bonsai** package must be installed and accessible), and performance will vary with the tuning grid; thus, results should be presented as outcomes of the specified benchmarking design rather than as general statements about the algorithms.

```
library(fastml)
library(bonsai)
library(NHANES)
library(dplyr)

set.seed(2025)

data("NHANES")

df <- NHANES %>%
  select(
    BPSysAve,
    Age, Gender, BMI, Race1, Education, Poverty,
    SleepHrsNight, PhysActive, Smoke100, TotChol, Diabetes
  ) %>%
  filter(!is.na(BPSysAve))

fm_bp <- fastml(
  data              = df,
  label             = "BPSysAve",
  algorithms        = c("elastic_net", "rand_forest", "lightgbm"),
  metric            = "rmse",
  impute_method     = "medianImpute",
  resampling_method = "cv",
  folds             = 5,
  tune_params       = list(
    elastic_net = list(
      glmnet = list(
        penalty = c(0.01, 0.1),
        mixture = c(0.0, 0.5, 1.0)  # ridge, elastic net, lasso
      )
    ),
    rand_forest = list(
      ranger = list(
        mtry  = c(2, 3, 5),
        min_n = c(20, 50),
        trees = c(100)
      )
    ),
    lightgbm = list(
      lightgbm = list(
        tree_depth    = c(20, 31),
        learn_rate    = c(0.05, 0.1)
      )
    )
  )
)

summary(fm_bp)
```

```
#>
#> ===== fastml Model Summary =====
#> Task: regression
#> Number of Models Trained: 3
#> Best Model(s): rand_forest (ranger) (rmse: 11.9474624)
#>
#> Performance Metrics (Sorted by rmse):
#>
#> ------------------------------------------------
#> Model         Engine    RMSE    R-squared  MAE
#> ------------------------------------------------
#> rand_forest*  ranger    11.947  0.533      8.715
#> elastic_net   glmnet    14.152  0.329      10.483
#> lightgbm      lightgbm  23.058  0.193      15.851
#> ------------------------------------------------
#> (*Best model)
#>
#> Best Model hyperparameters:
#>
#> Model: rand_forest (ranger)
#>   mtry: 5
#>   trees: 100
#>   min_n: 20
```

The resulting summary can be used to populate a regression comparison table, reporting RMSE and any additional metrics computed under the same resampling and tuning design. Any conclusions about which model "wins" should be tied to the declared tuning grid and evaluation plan: in particular, boosting methods can be sensitive to the search space, and a small grid may not reflect their best attainable performance.

These case studies demonstrate two applied patterns that recur in biomedical modeling: (i) comparative benchmarking across model families for diagnostic classification, and (ii) resampling-based regression benchmarking with explicit tuning budgets. The examples are intentionally written so that preprocessing decisions (e.g., exclusion of identifier fields), evaluation design (seed, resampling plan), and tuning scope are visible and reportable.

4 Conclusion

The R ecosystem provides increasingly capable machine-learning tooling, but correct evaluation still depends on how users structure preprocessing, resampling, and model selection. A common failure mode is leakage introduced by global data-dependent operations (e.g., imputation, scaling, feature construction) performed before resampling. Such issues can bias performance estimates even when outcome labels are never explicitly used (Kapoor and Narayanan, 2023).

fastml is motivated by reducing the practical risk of these errors through a constrained interface and guarded evaluation paths. When **fastml** executes its guarded resampling path, workflows are fitted within resamples so that data-dependent preprocessing is estimated on analysis splits and then applied to the corresponding assessment splits. This design can substantially reduce leakage risk in typical cross-validation setups (Section 2.1).

fastml also extends the same single-call workflow style to survival modeling. In addition to dispatching standard survival learners through established engines, **fastml** includes native implementations for selected survival methods (e.g., XGBoost AFT with interval bounds and a piecewise-exponential model implemented through flexsurv utilities). This enables side-by-side benchmarking of multiple survival model classes under one interface, while keeping the evaluation design (holdout vs. resampling) explicit in the analysis.

With respect to operational safety, **fastml** includes audit utilities intended to surface risky patterns in user-defined functions and recipe steps when `audit_mode = TRUE`. In the current implementation, recipe validation and audit logging can flag `.GlobalEnv`-style dependencies and I/O-related symbol usage. The audit layer is best characterized as warning/logging infrastructure rather than a hard enforcement mechanism that blocks all unsafe operations.

The empirical results presented in this paper (performance comparisons, model rankings, and usability examples) should be interpreted as outcomes of the experimental designs we executed, defined by specific datasets, preprocessing choices, tuning grids, random seeds, and resampling plans, rather than as properties guaranteed by the implementation. Likewise, any claims about reduced boilerplate relative to alternative frameworks should be treated as qualitative observations or externally measured comparisons, not as invariant guarantees.

Limitations. Several limitations should be noted. First, guarded resampling protects within the pipeline but not against upstream leakage (e.g., preprocessed inputs supplied by the user). Second, audit mode is observational rather than preventive for all patterns; it flags common risks but cannot intercept arbitrary unsafe operations. Third, survival resampling support is more constrained than for classification and regression tasks, with some method–resampling–tuning combinations explicitly unsupported. Fourth, portability of saved fastml objects depends on R and package versions, since fitted model objects may contain external pointers or version-dependent components. Finally, **fastml** does not currently support model stacking or ensemble combination; this is noted as a direction for future work.

Future work will focus on expanding the set of supported workflows while preserving evaluation clarity. Two concrete directions are (i) model combination strategies (e.g., stacking/super learning) implemented in a way that keeps resampling and leakage considerations explicit (van der Laan et al., 2007), and (ii) improved deployment ergonomics (e.g., generating standardized prediction wrappers and deployment-ready artifacts). These are planned extensions rather than current capabilities and will require careful design to maintain the package's emphasis on transparent evaluation and auditable outputs.

References

A. Barnwal, H. Cho, and T. D. Hocking. Survival regression with accelerated failure time model in XGBoost. *CoRR*, abs/2006.04920, 2020. URL https://arxiv.org/abs/2006.04920. [p10, 11]

Y. Bengio and Y. Grandvalet. No unbiased estimator of the variance of k-fold cross-validation. *Journal of Machine Learning Research*, 5(Sep):1089–1105, 2004. [p20]

C. Bergmeir and J. M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: https://doi.org/10.1016/j.ins.2011.12.028. [p5]

L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/a:1010933404324. [p21]

T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. KDD '16, Cornell University, 2016. URL https://medial-earlysign.github.io/MR_Wiki/attachments/5537821/5537823.pdf. [p21]

D. Eddelbuettel. *Seamless R and C++ Integration with Rcpp*. Springer New York, NY, first edition, 2013. [p13]

J. A. Foekens, H. A. Peters, M. P. Look, H. Portengen, M. Schmitt, M. D. Kramer, N. Brünner, F. Jänicke, M. E. M.-v. Gelder, S. C. Henzen-Logmans, W. L. J. van Putten, and J. G. M.

Klijn. The urokinase system of plasminogen activation and prognosis in 2780 breast cancer patients1. *Cancer Research*, 60(3):636–643, 2000. [p19]

H. Frick, F. Chow, M. Kuhn, M. Mahoney, J. Silge, and H. Wickham. *rsample: General Resampling Infrastructure*, 2025. URL https://CRAN.R-project.org/package=rsample. R package version 1.3.1. [p4]

M. Friedman. Piecewise exponential models for survival data with covariates. *The Annals of Statistics*, 10(1), 1982. doi: 10.1214/aos/1176345693. [p12]

T. Fryda, E. LeDell, N. Gill, S. Aiello, A. Fu, A. Candel, C. Click, T. Kraljevic, T. Nykodym, P. Aboyoun, M. Kurka, M. Malohlava, S. Poirier, W. Wong, L. Rehak, E. Eckstrand, B. Hill, S. Vidrio, S. Jadhawani, A. Wang, R. Peck, J. Gorecki, M. Dowle, Y. Tang, L. DiPerna, V. Maurerova, Y. Syzon, A. Valenta, M. Novotny, and H2O.ai. *h2o: R Interface for the H2O Scalable Machine Learning Platform*, 2020. URL https://CRAN.R-project.org/package=h2o. R package version 3.44.0.3. [p15]

J. Harrell, Frank E., R. M. Califf, D. B. Pryor, K. L. Lee, and R. A. Rosati. Evaluating the yield of medical tests. *JAMA*, 247(18):2543–2546, 1982. doi: 10.1001/jama.1982.03320430047030. [p10]

T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer, second edition, 2009. [p1, 2, 3]

E. Hvitfeldt and H. Frick. *censored: 'parsnip' Engines for Survival Models*, 2025. URL https://github.com/tidymodels/censored. R package version 0.3.3. [p15]

R. J. Hyndman and G. Athanasopoulos. *Forecasting: principles and practice*. OTexts, 2018. [p4, 5]

C. Jackson. flexsurv: A platform for parametric survival modeling in R. *Journal of Statistical Software*, 70(8):1â€"33, 2016. doi: 10.18637/jss.v070.i08. [p12]

J. D. Kalbfleisch and R. L. Prentice. *The Statistical Analysis of Failure Time Data*. Wiley, second edition, 2002. [p11]

S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804. [p1, 2, 25]

S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data*, 6(4), 2012. doi: 10.1145/2382577.2382579. [p2, 3]

G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf. [p23]

S. Korkmaz, D. Goksuluk, and E. Karaismailoglu. *fastml: Guarded Resampling Workflows for Safe and Automated Machine Learning in R*, 2025. URL https://CRAN.R-project.org/package=fastml. R package version 0.7.8. [p2]

M. Kuhn. Building predictive models in R using the caret package. *Journal of Statistical Software*, 28(5):1–26, 2008. doi: 10.18637/jss.v028.i05. [p1, 15]

M. Kuhn and K. Johnson. *Applied predictive modeling*, volume 26. Springer, 2013. [p4]

M. Kuhn and H. Wickham. *Tidymodels: a collection of packages for modeling and machine learning using tidyverse principles.*, 2020. URL https://www.tidymodels.org. [p1, 15]

M. Kuhn, H. Wickham, and E. Hvitfeldt. *recipes: Preprocessing and Feature Engineering Steps for Modeling*, 2025. URL https://CRAN.R-project.org/package=recipes. R package version 1.3.1. [p4]

R. A. Kyle, T. M. Therneau, S. V. Rajkumar, D. R. Larson, M. F. Plevak, J. R. Offord, A. Dispenzieri, J. A. Katzmann, and L. J. Melton. Prevalence of monoclonal gammopathy of undetermined significance. *New England Journal of Medicine*, 354(13):1362–1369, 2006. doi: 10.1056/NEJMoa054494. [p19]

M. Lang, M. Binder, J. Richter, P. Schratz, F. Pfisterer, S. Coors, Q. Au, G. Casalicchio, L. Kotthoff, and B. Bischl. mlr3: A modern object-oriented machine learning framework in R. *Journal of Open Source Software*, dec 2019. doi: 10.21105/joss.01903. [p1, 15]

J. F. Lawless. *Statistical Models and Methods for Lifetime Data*. Wiley, second edition, 2003. [p12]

F. Leisch and E. Dimitriadou. *mlbench: Machine Learning Benchmark Problems*, 2024. URL https://CRAN.R-project.org/package=mlbench. R package version 2.1-6. [p17]

R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. John Wiley & Sons, third edition, 2019. [p21]

C. L. Loprinzi, J. A. Laurie, H. S. Wieand, J. E. Krook, P. J. Novotny, J. W. Kugler, J. Bartel, M. Law, M. Bateman, and N. E. Klatt. Prospective evaluation of prognostic variables from patient-completed questionnaires. North Central Cancer Treatment Group. *Journal of Clinical Oncology*, 12(3):601–607, 1994. doi: 10.1200/JCO.1994.12.3.601. [p19]

S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf. [p22]

J. Luo, M. Schumacher, A. Scherer, D. Sanoudou, D. Megherbi, T. Davison, T. Shi, W. Tong, L. Shi, H. Hong, C. Zhao, F. Elloumi, W. Shi, R. Thomas, S. Lin, G. Tillinghast, G. Liu, Y. Zhou, D. Herman, Y. Li, Y. Deng, H. Fang, P. Bushel, M. Woods, and J. Zhang. A comparison of batch effect removal methods for enhancement of prediction performance using MAQC-II microarray gene expression data. *The Pharmacogenomics Journal*, 10(4): 278–291, 2010. doi: 10.1038/tpj.2010.57. [p5]

J. Ooms. The RAppArmor package: Enforcing security policies in R using dynamic sandboxing on Linux. *Journal of Statistical Software*, 55(7):1–34, 2013. doi: 10.18637/jss.v055.i07. [p14]

R. D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847. [p12]

R. Pruim. *NHANES: Data from the US National Health and Nutrition Examination Study*, 2025. URL https://CRAN.R-project.org/package=NHANES. R package version 2.1.0. [p23]

D. R. Roberts, V. Bahn, S. Ciuti, M. S. Boyce, J. Elith, G. Guillera-Arroita, S. Hauenstein, J. J. Lahoz-Monfort, B. Schröder, W. Thuiller, D. I. Warton, B. A. Wintle, F. Hartig, and C. F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: https://doi.org/10.1111/ecog.02881. [p5]

M. Rosenblatt, L. Tejavibulya, R. Jiang, S. Noble, and D. Scheinost. Data leakage inflates prediction performance in connectome-based machine learning models. *Nature Communications*, 15(1), 2024. doi: 10.1038/s41467-024-46150-w. [p1]

J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes. Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. *Proceedings of the Symposium on Computer Applications in Medical Care*, pages 261–265, 1988. [p17]

V. Stodden, F. Leisch, and R. D. Peng. *Implementing Reproducible Research*. CRC Press, 2014. [p13]

W. N. Street, W. H. Wolberg, and O. L. Mangasarian. Nuclear feature extraction for breast tumor diagnosis. In R. S. Acharya and D. B. Goldgof, editors, *Biomedical Image Processing and Biomedical Visualization*, volume 1905, pages 861 – 870. International Society for Optics and Photonics, SPIE, 1993. doi: 10.1117/12.148698. [p21]

I. E. Tampu, A. Eklund, and N. Haj-Hosseini. Inflation of test accuracy due to data leakage in deep learning-based classification of OCT images. *Scientific Data*, 9(1), 2022. doi: 10.1038/s41597-022-01618-6. [p1]

T. M. Therneau. *A Package for Survival Analysis in R*, 2024. URL https://CRAN.R-project.org/package=survival. R package version 3.8-3. [p10]

T. M. Therneau and P. M. Grambsch. *Modeling Survival Data: Extending the Cox Model*. Springer, New York, 2000. ISBN 0-387-98784-3. [p10]

A. Vabalas, E. Gowen, E. Poliakoff, and A. J. Casson. Machine learning algorithm validation with a limited sample size. *PLoS One*, 14(11):1–20, 2019. doi: 10.1371/journal.pone.0224365. [p2]

M. J. van der Laan, E. C. Polley, and A. E. Hubbard. Super learner. *U.C. Berkeley Division of Biostatistics Working Paper Series*, Working Paper 222, 2007. URL https://biostats.bepress.com/ucbbiostat/paper222. [p26]

H. Wickham. *Advanced R*. CRC Press, second edition, 2019. [p12]

G. Wilson, D. A. Aruliah, C. T. Brown, N. P. Chue Hong, M. Davis, R. T. Guy, S. H. D. Haddock, K. D. Huff, I. M. Mitchell, M. D. Plumbley, B. Waugh, E. P. White, and P. Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1):1–7, 2014. doi: 10.1371/journal.pbio.1001745. [p12]

H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 67(2):301–320, 2005. doi: 10.1111/j.1467-9868.2005.00503.x. [p23]

Selcuk Korkmaz
Trakya University
Faculty of Medicine, Department of Biostatistics, Balkan Campus
Edirne 22030, Turkiye
ORCiD: 0000-0003-4632-6850
selcukorkmaz@gmail.com

Dincer Goksuluk
Sakarya University
Faculty of Medicine, Department of Biostatistics, Korucuk Campus
Sakarya 54290, Turkiye
ORCiD: 0000-0002-2752-7668
dincergoksuluk@sakarya.edu.tr

Eda Karaismailoglu
University of Health Sciences
Faculty of Medicine, Department of Biostatistics, Gulhane Campus

Ankara 06010, Turkiye
ORCiD: 0000-0003-3085-7809
eda.karaismailoglu@sbu.edu.tr